Setting Up Simulation Environment and Manual Driving of MacAEV

ELECENG 3EY4: Electrical Systems Integration Project

Instructor: Shahin Sirouspour

Objective: In this week, you will install and set up the F1tenth Simulator package on the MacAEV on-board computer and will test its functionality in ROS. You will also modify the package so it can be used to manually drive the MacAEV with the Logitech F710 wireless joystick. Finally, you will experimentally tune the vehicle wheel odometry parameters for future use in autonomous driving.

Safety Instructions:

- Refer to the safety instructions on the course Avenue to Learn website and the course outline before attempting to carry out the activities set out in this document.
- The AEV must never by powered the LiPo battery except in the lab and under the supervision of a course TA.
- You must never remove the LiPo batteries from the lab rooms.
- You must only use the wall-mounted switching power supply to power the AEV for activities that can be performed outside the regular laboratory hours at home.
- When driving the AEV, you must ensure that it is always within you reach so you could safely stop it in case of an emergency.

General Instructions:

- In this document, the generic heading term “Activity” is used for actionable items as well as questions. These items must be specifically addressed in your written report.
- The symbol denotes an Activity that can be performed outside the laboratory at home prior to attending the lab, or after the lab if the activity requires answering questions based on your observations in the lab. You are strongly encouraged to complete these activities outside the laboratory hours to make efficient use of your lab time.

- The symbol denotes an Activity that must be carried out in the lab under TA supervision by powering the AEV from the LiPO battery.
- In addition to your written report, your submission should include copies of all files created or modified by you during the laboratory activities.

1 F1TENTH Racecar Simulator

The F1TENTH Racecar Simulator, an open-source ROS-based simulation environment, has been adapted to simulate the operation of the MacAEV in this course. This simulation environment allows you to debug, tune parameters, and evaluate the performance of your control algorithms before implementing them on the actual AEV. Additionally, you'll build the vehicle software control stack based on the simulation package, ensuring that the two systems share many common modules. This streamlines the transition from simulation to experiment.

Activity 1:

Before you install the simulator software stack, execute the following commands to 1) install the correct joystick driver and 2) install the correct VESC package for ROS Melodic.

```
$ cd ~/catkin_ws
$ sudo rm -r joystick_drivers # removes the incorrect version
$ cd ~/catkin_ws/src
$ git clone https://github.com/ros-drivers/joystick_drivers/ -b
melodic-devel
$ cd joystick_drivers
$ sudo rm -r wiimote # removes the Wiimote driver to avoid conflicts
$ cd ~/catkin_ws/src
$ git clone https://github.com/3EY4/vesc
$ cd ~/catkin_ws
$ catkin_make # this should successfully install the packages
```

Activity 2:

Install the simulator software stack; Download and build the F1TENTH Simulator from its source code on the GitHub repository here. The instructions for downloading and building the code are provided on the GitHub repository page. Note that you may have to install the following additional package dependencies:

tf2_geometry_msgs, ackermann_msgs, joy, map_server

Instructions for installing the dependencies can be found in the same repository. During the lab, demonstrate to your TA that you have completed this activity.

2 F1THENTH Simulator Code Structure

The F1tenth Simulator code is structured as shown in Fig. 1. The "Simulator" block implements the vehicle dynamics and kinematics. The block outputs, denoted as "Sensors", are *simulated* LiDAR scan data, Inertial Measurement Unit (IMU) data, and the vehicle wheel odometry data. The Simulator block receives the vehicle steering angle and linear velocity commands via the ROS topic "/drive" topic.

The "Mux" block passes on one of its inputs to "/drive" topic based on the value of the signal in topic "/mux". The value of this selector input is determined by the "Behavior Controller" block. This block decides the vehicle *Control Mode* based on the inputs from the Joystick or Keyboard and/or sensor observations. For example, the user may select to drive the vehicle manually via the Joystick in which case the Joystick commands are directly passed through the Mux block to the simulator. You can add your own driving algorithm code as a "New Planner" block and add a corresponding selection criterion to the "Behavior Controller" block. For more information about the F1TENTH simulator, please refer to its GitHub repository page here.

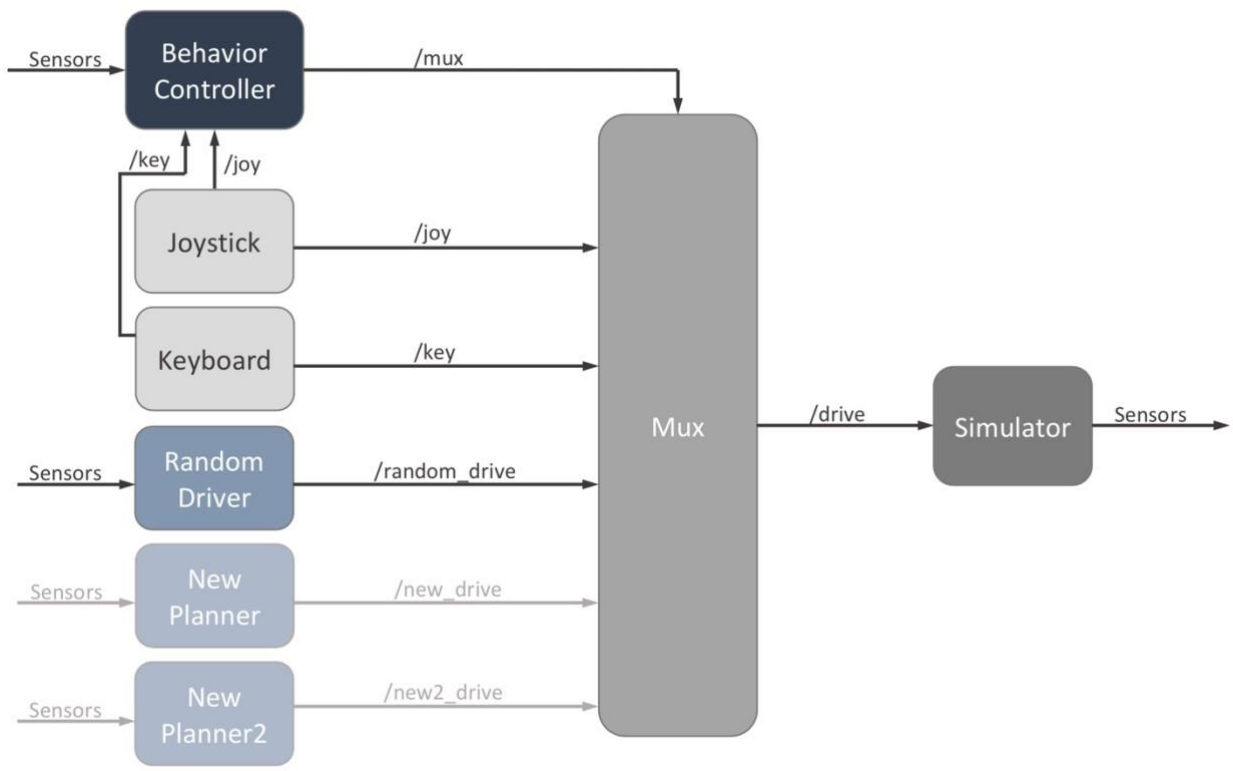

Fig. 1: F1TENTH Simulator Code Structure (image source here)

The simulator models the F1THENTH vehicle behavior based on the parameters specified in the file "…/src/f1tenth_simulator/params.yaml". The default values in the downloaded file are based on the hardware specifications of F1TENTH vehicle. You must replace this file with the one

with same name provided to you on Avenue to Learn. This would ensure that your simulation parameters will more closely reflect the specifications of the McMaster AEV, increasing consistency between your simulations and experiments.

Activity 3:

Replace the contents of the existing "params.yaml" with the one posted on the course website. The two parameters update_pose_rate and scan_beams in "params.yaml" file determine the LiDAR one full scan time (in seconds) and the number of LiDAR returns per one full scan. Ideally, you should set their values, so they reflect the specifications of the RPLiDAR scanner used in the actual vehicle. However, since the parameter update_pose_rate also sets the length of time step for solving the vehicle motion equations in discrete time, you should choose a smaller value for it to maintain numerical accuracy (e.g., 0.01 seconds). The RPLIDAR is set to run in the boost mode which returns 720 range readings in one full scan. This should be the value of scan_beams in your parameter file.

The file "simulator.launch" under "/launch" directory in the Simulator Package is a ROS *Launch* file. Launch files can be used to launch multiple ROS *nodes* simultaneously.

Activity 4:

What is a *node* in ROS?

Activity 5:

Briefly explain the role of each *node* launched by "simulator.launch" and relate them to their corresponding blocks in Fig. 1. Launch the Simulator by executing the following command:

```
$ roslaunch f1tenth_simulator simulator.launch
```

This should launch several nodes as well as the "rviz" visualization environment (see Fig. 2). If you press and release the LB button on the Logitech F710 Joystick the behavior controller selects the Joystick as the input device for the vehicle control. This would allow you to drive the vehicle inside the virtual environment using the left and right analog mini-sticks (see Fig. 3). Note that the LiDAR scan data is color coded by the distance of the laser returns from the scanner frame origin. By examining the content of the code provided to you, you should be able to figure out how the joystick action buttons are assigned for operating the vehicle in the simulator environment.

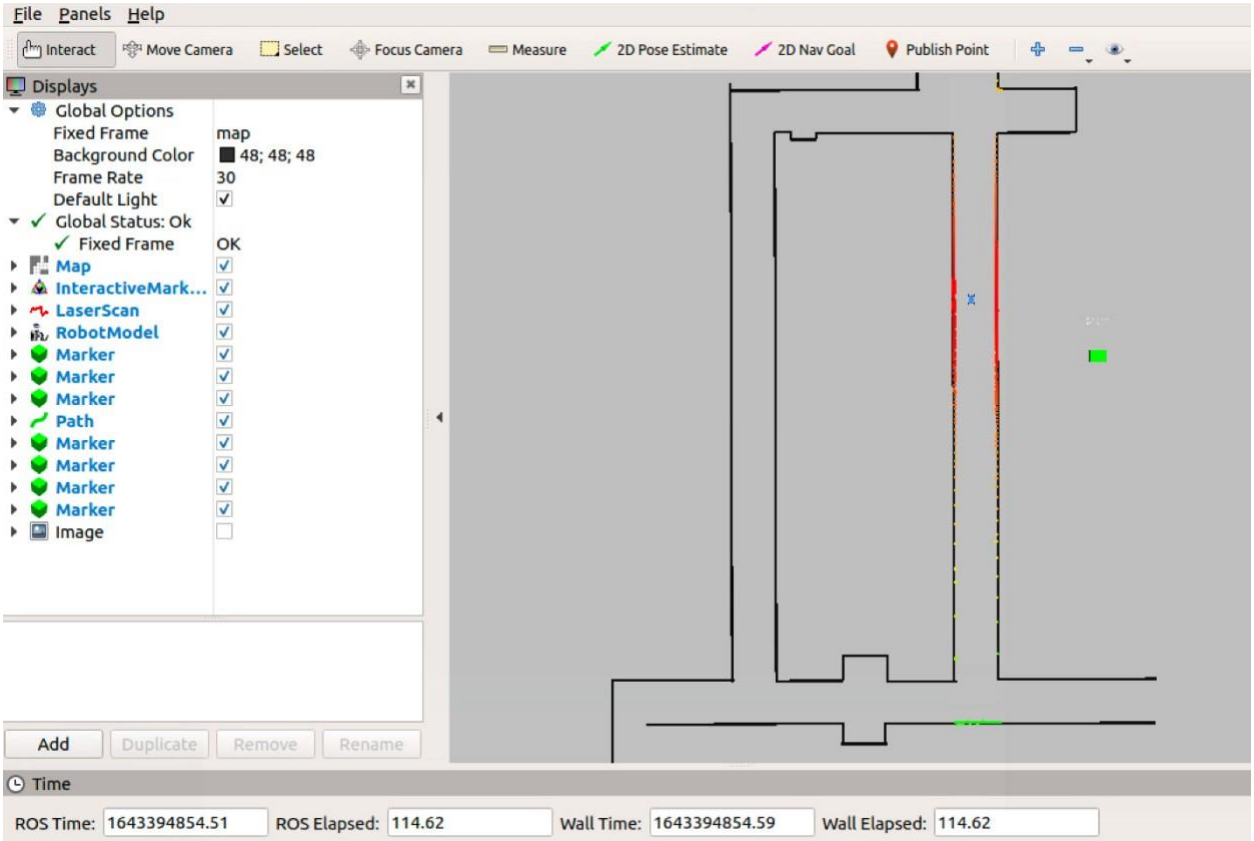

Figure 2: A screenshot of F1TENTH Simulator output in rviz

Figure 3: Logitech F710 Wireless Gamepad

Activity 6:

The ROS package *rqt_graph* is a visualization plug-in for ROS computation graph (see here for information). Open a terminal window and execute the following command:

```
$ roscore
```

Open another terminal window and run the following command:

```
$ rosparam set enable_statistics true
```

Re-run the simulator by launching "simulator.launch" file and execute the command below to run *rqt_graph* plug-in:

```
$ rosrun rqt_graph rqt_graph
```

You would see a graph like the one in Fig. 4. This is the ROS *computation graph*. It shows the *nodes* currently running, how they communicate through various *topics*, and the topic update rates. Note that you need to refresh the graph as you drive the vehicle in the simulation environment to see the most up-to-date numbers. You can switch the display type in your graph to "Nodes/Topics (all)" to get more information on the plot like the one in Fig. 5. Provide screenshots of the computation graphs in your report. Based on your computation graphs, what topics are published by the "/f1tenth_simulator" node? and what topics are the node subscribe to? Comment on the update rates of various topics in the computation graph and the factors that could influence these rates. Briefly explain the role of topics in ROS.

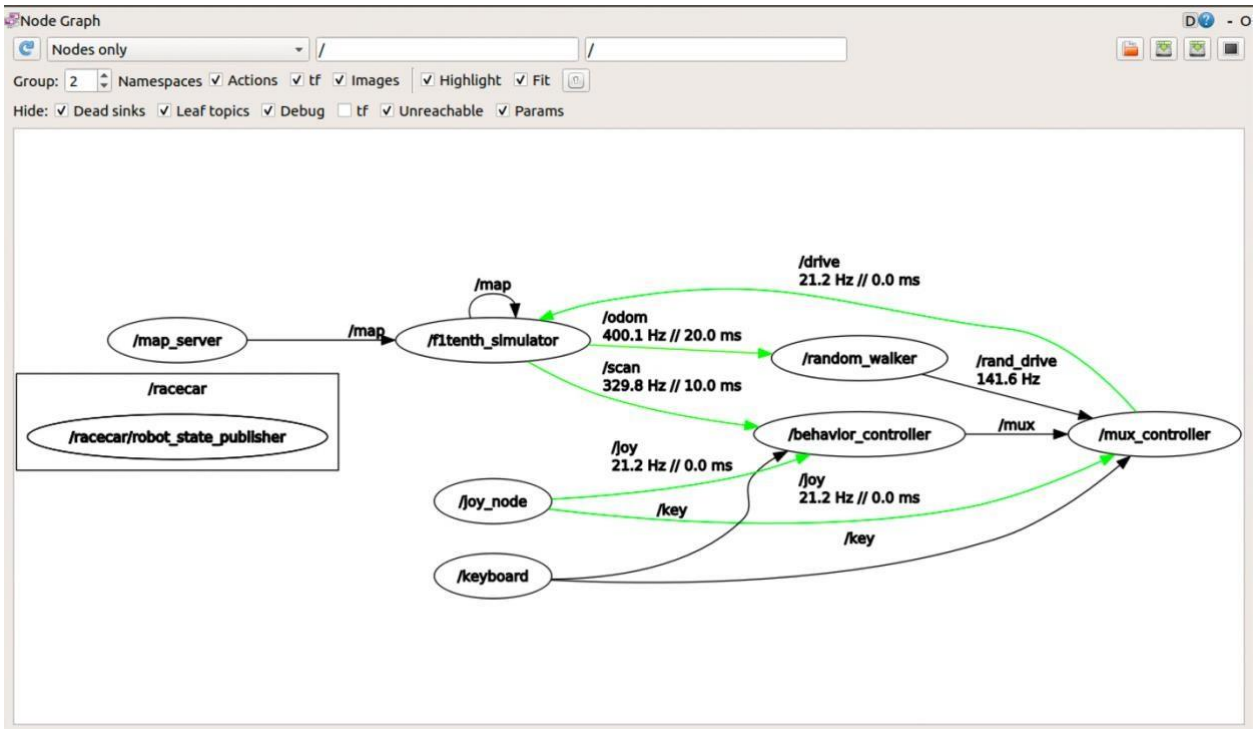

Figure 4: The ROS computation graph

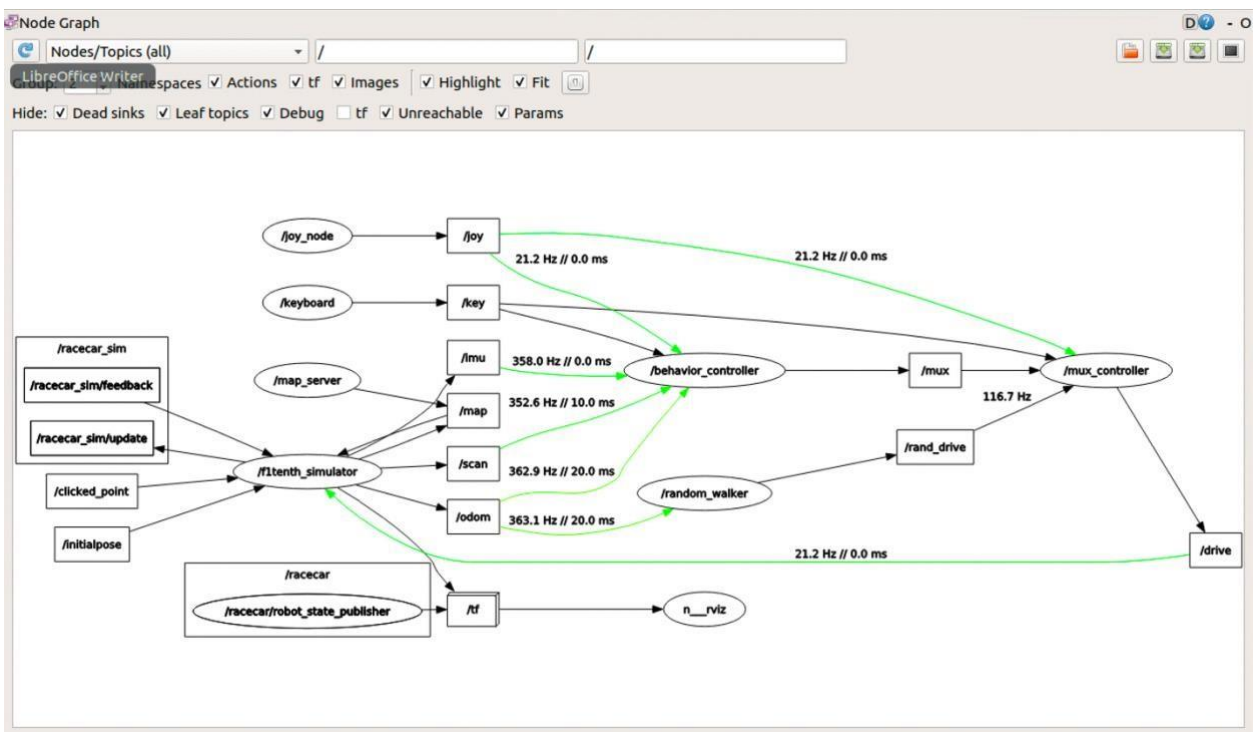

Figure 5: The ROS computation graph with additional details

The topic "/drive" contains the vehicle driving commands. The topic is of "ackermann_msgs/AckermannDriveStamped" type, which is a standard ROS time-stamped drive

command for robots with Ackermann steering. Execute the following command to obtain information on the topic "/drive":

```
$ rostopic info /drive
```

You should see that the data published on this topic is of "AckermannDrive" type, which is defined as:

```
## Driving command for a car-like vehicle using Ackermann steering.
#  $Id$

# Assumes Ackermann front-wheel steering. The left and right front
# wheels are generally at different angles. To simplify, the commanded
# angle corresponds to the yaw of a virtual wheel located at the
# center of the front axle, like on a tricycle.  Positive yaw is to
# the left. (This is *not* the angle of the steering wheel inside the
# passenger compartment.)
#
# Zero steering angle velocity means change the steering angle as
# quickly as possible. Positive velocity indicates a desired absolute
# rate of change either left or right. The controller tries not to
# exceed this limit in either direction, but sometimes it might.
#
float32 steering_angle          # desired virtual angle (radians)
float32 steering_angle_velocity # desired rate of change (radians/s)

# Drive at requested speed, acceleration and jerk (the 1st, 2nd and
# 3rd derivatives of position). All are measured at the vehicle's
# center of rotation, typically the center of the rear axle. The
# controller tries not to exceed these limits in either direction, but
# sometimes it might.
#
# Speed is the desired scalar magnitude of the velocity vector.
# Direction is forward unless the sign is negative, indicating reverse.
#
# Zero acceleration means change speed as quickly as
# possible. Positive acceleration indicates a desired absolute
# magnitude; that includes deceleration.
#
# Zero jerk means change acceleration as quickly as possible. Positive
# jerk indicates a desired absolute rate of acceleration change in
# either direction (increasing or decreasing).
#
float32 speed                   # desired forward speed (m/s)
float32 acceleration            # desired acceleration (m/s^2)
float32 jerk                    # desired jerk (m/s^3)
```

Refer to the lecture notes for information on the vehicle kinematics model and Ackermann steering.

Activity 7:

Using the ROS command "rostopic", echo the "/drive" topic onto your terminal screen while driving the vehicle in the simulator. Which node publishes to the "/drive" topic and at what rate? Which fields of the "/drive" topic are updated and what do they represent? Provide your answers in your written report.

3 Modifying Simulator Code for Manual Driving

We now set up the vehicle for manual driving with the Logitech F710 Joystick. To accomplish this goal, we will modify and build on the F1TENTH simulator software stack.

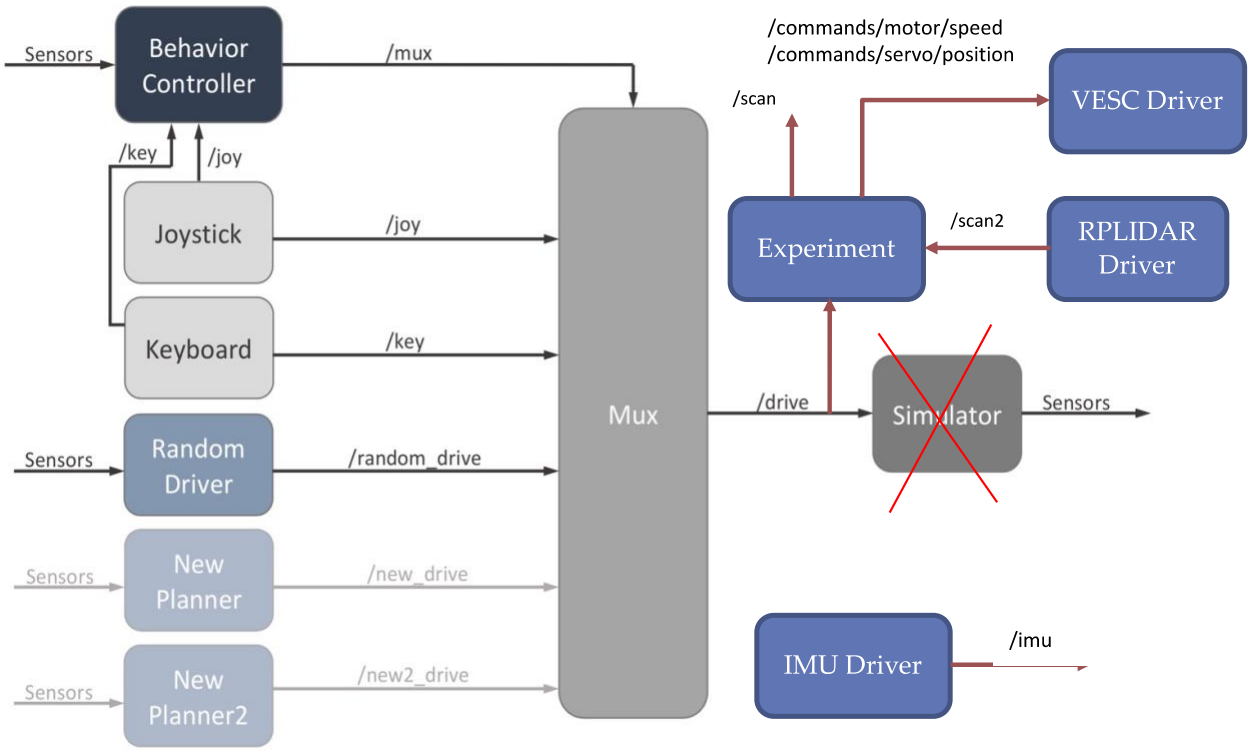

Figure 6: Revised Software Structure for Controlling McMaster AEV

Refer to Fig. 6 for the revised software stack that is used to control the McMaster AEV. The Simulator block is removed, and replaced with four new blocks: VESC Driver, RPLIDAR Driver, IMU Driver, and Experiment. The blocks interface with the actuators and sensors of the actual vehicle. The new block "Experiment" is added to *translate* the output "/drive" command of the "Mux" block into commands suitable for the motor drive via VESC driver.

Recall that the "/drive" topic is an Ackermann steering command comprised of a linear vehicle velocity command v_s in (m/s) and a vehicle steering command δ in (radians). This data in

from the "/drive" topic is separated into two commands, i.e., "/commands/motor/speed" for motor speed and "/commands/servo/position" for steering servo position; these are passed on to the ROS-based VESC driver.

Remark: Throughout the rest of the project, we will be controlling the motor in *speed mode*.

Activity 8:

In your report, discuss the various modes of motor control (e.g., speed, torque, etc.) and comment on their suitability for use in electrical vehicles in manual and self-driving modes.

The RPLIDAR driver publishes the laser scan data over the topic "/scan2". This data is rearranged and published over the topic "/scan" to ensure consistency between the simulator and the actual LiDAR over how the laser scan data is reported (more on this later in this document).

Activity 9:

In the next few steps, we will modify the existing simulator software stack by adding and building a new node to implement the functionality of the block "Experiment":

1. First create an empty file called "experiment.cpp" in the directory "…/ /src/f1tenth_simulator/node". Then, copy the contents of the "experiment.cpp" file provided on Avenue to learn into this empty file. This is C++ source code for the new node. Change the file to executable by:
   ```
   $ sudo chmod +x experiment.cpp
   ```
2. Add the line **"<run_depend>vesc_driver</run_depend>"** to the file "…/src/f1tenth_simulator/package.xml".
3. Execute the following command:
   ```
   $ sudo cp -r ~/catkin_ws/devel/include/vesc_msgs/
   ~/catkin_ws/src/f1tenth_simulator/include/
   ```
4. Rebuild the simulator package by executing the "catkin_make" command from the root catkin workspace directory. This should build the executable for the new node.
5. Create an empty launch file called "experiment.launch" in the launch directory of the simulator package. Then, copy the contents of the "experiment.launch" file provided on Avenue to Learn into this new launch file.
6. During the lab, demonstrate to your TA that you have completed this activity.

Activity 10:

In your report, explain the purpose of the following two functions in the file "experiment.cpp". In the function "laser_callback()", do you anticipate a potential issue arising depending on whether the number of data points in a single scan is even or odd number? Explain. Modify the code to address this potential bug. Include the modified code as part of your lab submission.

```
void publish_driver_command( const ros::TimerEvent&){

   double desired_delta = desired_steer_ang-last_servo;
   double clipped_delta = std::max(std::min(desired_delta,
max_delta_servo), -max_delta_servo);
   double smoothed_servo = last_servo + clipped_delta;
   last_servo = smoothed_servo;
   servo_msg.data=smoothed_servo;

   double desired_rpm = desired_speed-last_rpm;
   double clipped_rpm = std::max(std::min(desired_rpm, max_delta_rpm), max_delta_rpm);
   double smoothed_rpm = last_rpm + clipped_rpm;
   last_rpm = smoothed_rpm;
   erpm_msg.data=smoothed_rpm;

   servo_pub.publish(servo_msg);
   erpm_pub.publish(erpm_msg);

}

void laser_callback(const sensor_msgs::LaserScan & msg) {

   int n1=msg.ranges.size()/2;
   std::vector<float> scan_ranges=msg.ranges;
   std::vector<float> scan_intensities=msg.intensities;
   for (int i=0;i<n1;i++){
       scan_ranges[i]=msg.ranges[n1+i];
       scan_ranges[n1+i]=msg.ranges[i];
       scan_intensities[i]=msg.intensities[n1+i];
       scan_intensities[n1+i]=msg.intensities[i];
```

```
    }

    // Publish lidar information
    sensor_msgs::LaserScan scan_msg;
    scan_msg.header.stamp = msg.header.stamp;
    scan_msg.header.frame_id = msg.header.frame_id;
    scan_msg.angle_min = msg.angle_min+3.141592;
    scan_msg.angle_max = msg.angle_max+3.141592;
    scan_msg.angle_increment = msg.angle_increment;
    scan_msg.range_max = msg.range_max;
    scan_msg.ranges = scan_ranges;
    scan_msg.intensities = scan_intensities;
    lidar_pub.publish(scan_msg);
}
```

4 Manual Driving

You are now ready to drive the vehicle manually using the Joystick.

Activity 11:

Open the "params.yaml" file and ensure the value of the parameter "max_speed" is set to a value between 1-1.5 m/s. This would limit the vehicle speed command to a safe maximum value. Similarly, the "max_steering_angle" should be set to a value around "0.41" radians. Put the F710 Joystick in "x" mode if it is not already. To find out if things are running smoothly, place the vehicle on the black foam block which was provided as part of the AEV kit. During the lab, demonstrate to your TA that you have completed this activity.

Activity 12:

Start operating the vehicle by the following launch command:

```
$ roslaunch f1tenth_simulator experiment.launch
```

Report the ROS topics associated with the VESC ROS driver. Examine the rate at which the motor speed and the servo steering angle commands are published to their respective topics. What is the rate of execution for the VSEC driver node "/vesc_driver_node"? What factors affect these rates? (you can use the command `rostopic list`). Your findings should be included in your written report.

Press and release the LB on the Joystick once to enable manual driving. Use the two mini-sticks to issue speed and steering commands to the vehicle. If the vehicle is responding as expected,

you can place it on the ground and start driving it around. Make sure that you stay close enough to the vehicle so you would not lose communication between the F710 Joystick and Jetson Nano. After finishing your drive, shut down the program by pressing ctrl-c in the command window. During the lab, demonstrate to your TA that you have completed this activity.

5 Calibrating Ackermann-to-VESC Command Parameters

In "Velocity Mode", the VESC driver accepts a reference angular velocity command $\omega_e$in (erpm) (electrical rpm) (/commands/motor/velocity), and a reference servo position command δ_s (/commands/servo/position). However, the Mux block outputs Ackermann drive commands, with a linear vehicle velocity v_s in m/s and the servo steering angle δ. First, we need to convert the linear speed command v_s (m/s) to the corresponding angular velocity ω_e (erpm), i.e.,

$$\omega_e = k_\omega v_s \tag{1}$$

The value of k_ω can be computed from the specifications of the vehicle transmission system and the electric motor using the following equation:

$$k_\omega = \frac{15 p g_r}{\pi r_\omega} \tag{2}$$

Here p is the number of the motor poles, g_r is the final drive ratio of the vehicle transmission, and r_ω is the radius of the vehicle wheel.

Activity 13:

Derive the expression in (2) for k_ω. Include your derivations in the report.

The values of the parameters for the McMaster AEV are $p = 4$, $g_r = 10.83$, $r_\omega = 0.065$ m, which yield $k_\omega = 3182.12$. This should be the value of the variable "speed_to_erpm_gain" in the file "params.yaml".

The steering angle δ (rad) is related to servo command δ_s according to the affine linear equation:

$$\delta_s = k_\delta \delta + \delta_0 \tag{3}$$

You have been provided with nominal values for δ_0, k_δ in the file "params.yaml". These values are derived from calibration experiments of a different vehicle. In the next few activities, you will be recalibrating these parameters to correct for the slight variations that exist from vehicle to vehicle.

Activity 14:

You need to first fine tune the offset value δ_0. Simply drive the vehicle in a straight line without giving any steering command. Notice if the vehicle drifts to left or right. If that is the case, make a small adjustment to δ_0 based on the direction of the drift. Repeat this refinement process until the vehicle no longer drifts away from a straight path. During the lab, demonstrate to your TA that you have completed this activity.

Activity 15:

Referring to the model in Fig. 7, the vehicle would move on a circular path with radius r when given a constant steering angle δ.

$$\tan\delta = \frac{l}{r} \ , r = \frac{l}{\tan\delta} \tag{4}$$

where l is the vehicle wheelbase. This formula provides a procedure for tuning up the value of k_δ.

Place the vehicle on the ground and mark the origin of its base_link frame, as shown in Fig. 7, as the starting position. You should mark the straight line that passes through that origin and the center of the turn. While the AEV is stationary, apply a maximum steering command to the vehicle by pushing the mini-stick all the way in the intended turn direction. Start driving forward until the vehicle completes half a circle and crosses over this line. The distance between the starting point and where the center of the rear axle crosses the line will have a value of $2r$. During the lab, demonstrate to your TA that you have completed this activity.

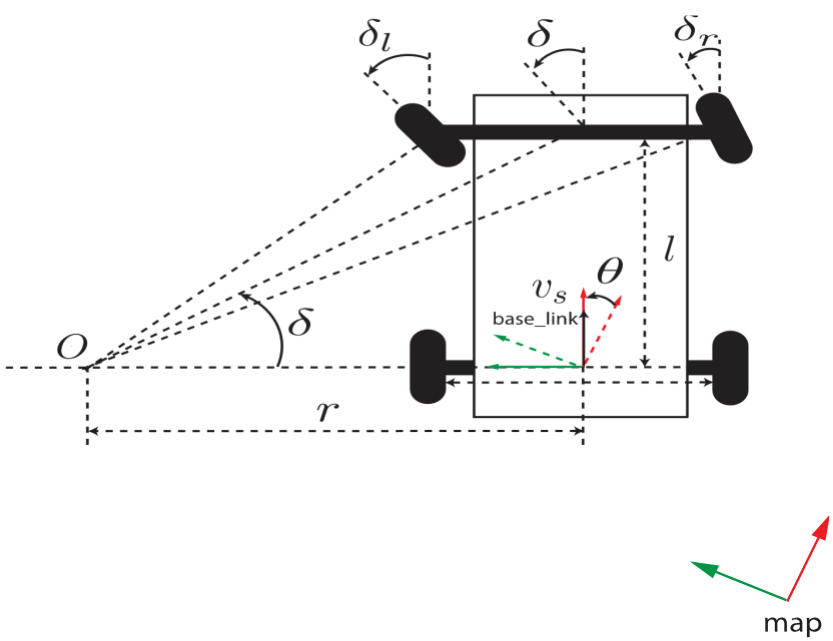

Figure 7: Vehicle driving model, also known as bicycle model

The desired steering angle, δ_d, can be obtained from the corresponding field in the published topic "/drive". The actual steering angle δ is given by:

$$\delta = \tan^{-1}\left(\frac{l}{r}\right) \tag{5}$$

The following equation can be used to re-calibrate the gain value in the steering equation. Let the current value of this gain (from "parameter.yaml" file) be denoted by $\bar{k}_\delta$. Then,

$$\delta_s = \bar{k}_\delta \delta_d + \delta_0 = k_\delta \delta + \delta_0 \tag{6}$$

or

$$k_\delta = \frac{\delta_d}{\delta}\,\bar{k}_\delta \tag{7}$$

Activity 16:

Use the results of calibration experiment to re-calculate the value of k_δ. Update the value of this parameter in "params.yaml". Include your calculations in your report.

Activity 17:

Examine the following line from the file "experiment.cpp" and explain what it does:

```
update_command = n.createTimer(ros::Duration(1.0/driver_smoother_rate),
                               &RacecarExperiment::publish_driver_command, this);
```

Execute the following command:

```
$ rostopic hz /commands/motor/speed
```

This should help you in answering the above question. Include your observation and answer in your report.

Activity 18:

In your report, provide one use case for each of the following ROS commands based on the activities of this lab:

```
$ rosrun
$ roslaunch
$ rosnode
$ rostopic
```